

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ
“ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”

Кафедра “Програмної інженерії та інтелектуальних технологій управління”

ЗВІТ

З лабораторної роботи №1
За темою: «**Робота зі вказівниками, рядками та файлами**»
Варіант 13

Виконав студент
Групи КН-224Б
Макаренко Максим Олександрович

Перевірів
Василенко Артем Вікторович

Харків-2025

Хід роботи

1.1 Виведення таблиці значень функції

Створити окрему одиницю трансляції, в якій реалізувати функцію виведення таблиці значень певної функції з визначеним кроком. Параметри функції – початок, кінець інтервалу, крок і вказівник на функцію, значення якої будуть виведені в таблиці. У заголовному файлі описати тип вказівника на функцію і прототип функції виведення таблиці значень. У файлі реалізації визначити функцію виведення таблиці значень.

В іншій одиниці трансляції розташувати функції, для яких слід вивести значення у таблиці, а також функцію `main()`, в якій здійснюється виведення таблиць значень не менш ніж двох різних функцій. Одна з функцій для тестування – функція, визначена в індивідуальному завданні першої лабораторної роботи попереднього семестру та реалізована в завданні четвертої лабораторної роботи попереднього семестру.

`main.cpp`

```
#include "table_printer.h"
#include <iostream>
#include <cmath>

// First test function (given)
double function1(double x) {
    double result = 0.0;
    int n = 2;
    if (x <= 0) {
        result = x;
        for (int i = 0; i < n; ++i) {
            result *= (pow(i, 3) + x);
        }
    } else {
        result = 0.0;
        for (int i = 1; i < n; ++i) {
            for (int j = 0; j < n; ++j) {
                result += x / (i + j);
            }
        }
    }
}
```

```

    }
    return result;
}

// // Second test function
double function2(double x) {
    if (x < 0) {
        return NAN;
    }
    if (x == 0) {
        return 1;
    }
    double result = 1;
    for (int i = 1; i <= x; ++i) {
        result *= i;
    }
    return result;
}

void readData(double& a, double& b, double& h) {
    std::cout << "Введіть початок інтервалу a: ";
    std::cin >> a;

    std::cout << "Введіть кінець інтервалу b: ";
    std::cin >> b;

    do {
        std::cout << "Введіть крок h (h > 0): ";
        std::cin >> h;
        if (h <= 0) {
            std::cout << "Крок h повинен бути більше 0. Спробуйте ще раз.\n";
        }
    } while (h <= 0);
}

int main() {

```

```

double a, b, h;

readData(a, b, h);

std::cout << "Testing Function 1 (given function):" << std::endl;
printFunctionTable(a, b, h, function1);

std::cout << "\nTesting Function 2 (count factorial):" << std::endl;
printFunctionTable(a, b, h, function2);

return 0;
}

```

table_printer.cpp

```

#include "table_printer.h"
#include <iostream>
#include <iomanip>

void printFunctionTable(double start, double end, double step, FunctionPtr func) {
    std::cout << std::setw(10) << "Start" << std::setw(10) << "End"
        << std::setw(10) << "Step" << std::setw(10) << "x"
        << std::setw(15) << "Result" << std::endl;
    std::cout << std::string(55, '-') << std::endl;

    for (double x = start; x <= end; x += step) {
        std::cout << std::fixed << std::setprecision(4)
            << std::setw(10) << start
            << std::setw(10) << end
            << std::setw(10) << step
            << "\033[32m"
            << std::setw(10) << x
            << std::setw(15) << func(x)
            << "\033[0m"
            << std::endl;
    }
}

```

```
std::cout << std::endl;  
}
```

table_printer.h

```
#ifndef TABLE_PRINTER_H  
#define TABLE_PRINTER_H  
  
typedef double (*FunctionPtr)(double);  
  
void printFunctionTable(double start, double end, double step, FunctionPtr func);  
  
#endif  
  
}
```

Додатковий опис:

1.Результат роботи:

```

● maksim@192 1.1 % ./program
Введіть початок інтервалу a: 1
Введіть кінець інтервалу b: 6
Введіть крок h (h > 0): 1
Testing Function 1 (given function):

```

Start	End	Step	x	Result
1.0000	6.0000	1.0000	1.0000	1.5000
1.0000	6.0000	1.0000	2.0000	3.0000
1.0000	6.0000	1.0000	3.0000	4.5000
1.0000	6.0000	1.0000	4.0000	6.0000
1.0000	6.0000	1.0000	5.0000	7.5000
1.0000	6.0000	1.0000	6.0000	9.0000

```

Testing Function 2 (count factorial):

```

Start	End	Step	x	Result
1.0000	6.0000	1.0000	1.0000	1.0000
1.0000	6.0000	1.0000	2.0000	2.0000
1.0000	6.0000	1.0000	3.0000	6.0000
1.0000	6.0000	1.0000	4.0000	24.0000
1.0000	6.0000	1.0000	5.0000	120.0000
1.0000	6.0000	1.0000	6.0000	720.0000

Рисунок 1 — результат коректної роботи програми

1.2 Індивідуальне завдання

Написати програму, яка реалізує перебір значень на визначеному інтервалі з певним кроком з метою пошуку деякого значення відповідно до індивідуального завдання, (Найбільший корінь другої похідної функції). Необхідне значення може бути знайдено шляхом отримання проміжних значень функції (або першої / другої похідної). Слід використати вказівник на функцію, для якого визначити typedef.

Сирцевий код повинен бути розділений на дві одиниці трансляції. Перша одиниця трансляції буде представлена заголовним файлом і файлом реалізації. Визначення typedef, а також прототип функції пошуку потрібного значення, повинні бути розташовані в заголовному файлі. Визначення цієї функції слід здійснити у файлі реалізації. Функція для перевірки працездатності програми, а також функція main(), повинні бути розташовані в іншій одиниці трансляції.

Слід перевірити працездатність програми не менш ніж на двох довільних функціях. Одна з функцій може бути стандартною.

Корінь – це значення аргументу, в якій функція повертає нуль.

Примітка: Для обчислення першої похідної $y'(x)$ можна використати таку формулу: $y'(x) = (y(x + \Delta x) - y(x)) / \Delta x$,

Де Δx деяке невеличке значення, наприклад 0.0000001.

main.cpp

```
#include "root_finder.h"
#include <iostream>
#include <iomanip>
#include <cmath>

// Test function 1: polynomial function  $x^3 - 2x^2 - x + 2$ 
// Its second derivative is  $6x - 4$ 
// The root of second derivative is at  $x = 2/3$ 
double testFunction1(double x) {
    return pow(x, 3) - 2 * pow(x, 2) - x + 2;
}

// Test function 2:  $\sin(x) + x^2$ 
```

```

// Its second derivative is -sin(x) + 2
double testFunction2(double x) {
    return sin(x) + pow(x, 2);
}

void printResult(const char* functionName, double root) {
    std::cout << "\nНайбільший корінь другої похідної для " << functionName << ": "
        << std::fixed << std::setprecision(6) << root << std::endl;
}

int main() {

    double start = -5.0;
    double end = 5.0;
    double step = 0.01;

    double root1 = LargestRoot(testFunction1, start, end, step);
    printResult("x^3 - 2x^2 - x + 2", root1);

    double root2 = LargestRoot(testFunction2, start, end, step);
    printResult("sin(x) + x^2", root2);

    return 0;
}

```

root_finder.cpp

```

#include "root_finder.h"
#include <cmath>

// first derivative (provided formula)
double firstDerivative(FunctionPtr func, double x, double dx = 1e-7) {
    return (func(x + dx) - func(x)) / dx;
}

// second derivative
double secondDerivative(FunctionPtr func, double x, double dx = 1e-7) {
    return (firstDerivative(func, x + dx) - firstDerivative(func, x)) / dx;
}

```



```

}

double LargestRoot(FunctionPtr func, double start, double end, double step, double epsilon) {
    double largestRoot = start;
    bool foundRoot = false;

    // Iterate through the interval
    for (double x = start; x <= end; x += step) {
        double y2 = secondDerivative(func, x);
        double y2_next = secondDerivative(func, x + step);

        // Check if second derivative changes sign (crosses zero)
        if (std::abs(y2) < epsilon || (y2 * y2_next <= 0)) {
            if (!foundRoot || x > largestRoot) {
                largestRoot = x;
                foundRoot = true;
            }
        }
    }

    return foundRoot ? largestRoot : start;
}

```

root_finder.h

```

#ifndef ROOT_FINDER_H
#define ROOT_FINDER_H

// pointer type for functions
typedef double (*FunctionPtr)(double);

double LargestRoot(FunctionPtr func, double start, double end, double step, double epsilon = 1e-7);

#endif

```

Додатковий опис:

1.Результат роботи:

```
● maksim@192 1.2 % ./program  
Найбільший корінь другої похідної для  $x^3 - 2x^2 - x + 2$ : 0.670000  
Найбільший корінь другої похідної для  $\sin(x) + x^2$ : -5.000000  
✧ maksim@192 1.2 %
```

Рисунок 2 — результат коректної роботи програми

1.3 Робота з масивом вказівників на функції

В окремому просторі імен описати синонім типу вказівника на функцію, яка приймає два аргументи типу `double` та повертає результат типу `double`. В тому ж просторі імен реалізувати функцію з типом результату `void`, яка отримує як параметр раніше описаний вказівник на функцію та значення двох аргументів і виводить на екран значення аргументів і значення функції.

Поза створеним простором імен визначити функцію, яка приймає два аргументи типу `double` та повертає суму других степенів аргументів.

У функції `main()` створити масив вказівників на функції описаного раніше типу. В масиві розташувати вказівники на стандартні функції заголовного файлу `cmath`, які отримують два аргументи, а також вказівник на раніше створену функцію обчислення суми других степенів. Прочитати з клавіатури значення двох аргументів і в циклі вивести аргументи й функцію для всіх вказівників на функції з масиву.

Рекомендовані функції заголовного файлу `cmath`, які отримують два аргументи типу `double` та повертають результат типу `double`:

- `pow()` – довільний степінь числа;
- `hypot()` – величина гіпотенузи для двох вказаних катетів прямокутного трикутника;
- `fmax()` – максимальне з двох значень;
- `fmin()` – мінімальне з двох значень;

Продемонструвати різні способи підключення елементів простору імен.

`main.cpp`

```
#include "mathfunc.h"
#include <iostream>
#include <cmath>
#include <iomanip>

using MathOperations::MathFunction;

int main() {
    MathFunction functions[] = {
        std::pow,
        std::hypot,
        std::fmax,
```

```

    std::fmin,
    sumSquares
};

const char* functionNames[] = {
    "pow",
    "hypot",
    "fmax",
    "fmin",
    "sumSquares"
};

double x, y;
std::cout << "Введіть значення x: ";
std::cin >> x;
std::cout << "Введіть значення y: ";
std::cin >> y;

std::cout << "\nРезультати обчислень:\n";
std::cout << std::string(50, '-') << std::endl;

// 1. explicit access qualification
std::cout << "Спосіб 1 - Явна кваліфікація:\n";
for (int i = 0; i < 5; ++i) {
    std::cout << std::setw(12) << functionNames[i];
    MathOperations::printResult(functions[i], x, y);
}

std::cout << std::string(67, '-') << std::endl;

// 2. using-directive
std::cout << "Спосіб 2 - Using-директива:\n";
{
    using namespace MathOperations;
    for (int i = 0; i < 5; ++i) {
        std::cout << std::setw(12) << functionNames[i];
        printResult(functions[i], x, y);
    }
}

```

```

}

std::cout << std::string(67, '-') << std::endl;

// 3. using-declaration
std::cout << "Списіб 3 - Using-оголошення:\n";
{
    using MathOperations::printResult;
    std::cout << std::setw(12) << "sumSquares";
    printResult(sumSquares, x, y);
}

return 0;
}

```

mathfunc.cpp

```

#include "mathfunc.h"
#include <iostream>
#include <iomanip>

namespace MathOperations {
    void printResult(MathFunction func, double x, double y) {
        std::cout << " | x = " << std::setw(8) << std::fixed << std::setprecision(2) << x
            << ", y = " << std::setw(8) << y
            << " | Результат = " << std::setw(10) << func(x, y) << std::endl;
    }
}

double sumSquares(double x, double y) {
    return x * x + y * y;
}

```

mathfunc.h

```
#ifndef MATHFUNC_H
#define MATHFUNC_H

namespace MathOperations {
    typedef double (*MathFunction)(double, double);

    void printResult(MathFunction func, double x, double y);
}

double sumSquares(double x, double y);

#endif
```

Додатковий опис:

1.Результат при коректній роботі умови:

```
● maksim@192 1.3 % ./program
Введіть значення x: 1
Введіть значення y: 4

Результати обчислень:
-----
Спосіб 1 – Явна кваліфікація:
      pow | x =    1.00, y =    4.00 | Результат =    1.00
     hypot | x =    1.00, y =    4.00 | Результат =    4.12
      fmax | x =    1.00, y =    4.00 | Результат =    4.00
      fmin | x =    1.00, y =    4.00 | Результат =    1.00
sumSquares | x =    1.00, y =    4.00 | Результат =   17.00
-----
Спосіб 2 – Using-директива:
      pow | x =    1.00, y =    4.00 | Результат =    1.00
     hypot | x =    1.00, y =    4.00 | Результат =    4.12
      fmax | x =    1.00, y =    4.00 | Результат =    4.00
      fmin | x =    1.00, y =    4.00 | Результат =    1.00
sumSquares | x =    1.00, y =    4.00 | Результат =   17.00
-----
Спосіб 3 – Using-оголошення:
sumSquares | x =    1.00, y =    4.00 | Результат =   17.00
✧ maksim@192 1.3 %
```

Рисунок 3 — результат коректної роботи програми

Висновок:

У ході виконання лабораторної роботи було розглянуто роботу з вказівниками на функції та заголовними файлами в мові програмування C++. Основні завдання включали:

- Виведення таблиці значень функції: Було створено окрему одиницю трансляції, де реалізовано функцію для виведення таблиці значень функції на заданому інтервалі з певним кроком. Використання вказівників на функції дозволило універсалізувати підхід до обчислення значень різних функцій.
- Індивідуальне завдання: Реалізовано програму для пошуку певного значення на інтервалі з використанням вказівників на функції. Завдання включало обчислення похідних та пошук екстремумів, коренів або інших значень відповідно до варіанту. Використання typedef для вказівників на функції спростило роботу з ними.
- Робота з масивом вказівників на функції: Було створено масив вказівників на функції, що дозволило викликати різні функції (стандартні та користувацькі) через цикл. Це продемонструвало можливість динамічного вибору функцій під час виконання програми.
- Використання заголовних файлів: Заголовні файли були використані для оголошення типів, прототипів функцій та констант, що дозволило розділити код на модулі та уникнути дублювання коду. Використання "стражів включення" (#ifndef, #define, #endif) запобігло проблемам з багаторазовим включенням заголовних файлів.
- Простори імен: Використання просторів імен дозволило уникнути конфліктів імен та логічно структурувати код. Було продемонстровано різні способи підключення елементів просторів імен (using, ::).

Ця робота закріпила навички роботи з вказівниками на функції, заголовними файлами та просторами імен, що є важливими для розробки модульних та універсальних програм на C++.